

BPS 语言标准(Basic Programming Script Language Standard)

文档版本：Basic Programming Script Language Standard 2026（修订 2）

简称：BPS-26.2

修订日期：2026 年 7 月 21 日

参考实现：Basic Programming Script 0.0.2.3（以下简称 BPS）

1. 概述

Basic Programming Script 是一门轻量级解释型脚本语言，专为网络自动化任务设计。本标准定义了 BPS 语言的语法、语义及运行时行为。

1.1 设计目标

- （1）语法简洁，学习成本低
- （2）原生 HTTP 客户端/服务端支持
- （3）无需外部依赖

1.2 标准符合性

本实现符合 BPS-26.2 标准的所有要求。

2. 词法结构

2.1 字符集

- （1）支持 ASCII 字符集
- （2）字符串字面量中支持 Unicode

2.2 注释

类型	语法	示例
单行注释	!!	!!这是注释
多行注释	/! ... !/	/! 多行 !/

2.3 关键字

2.3.1 变量与流程控制

序号	关键字	说明
1	DEFINE	定义变量
2	IF	条件判断
3	ELSE	否则
4	WHILE	循环
5	RETURN	返回值
6	FUNCTION	函数定义
7	MAIN	主函数
8	WAIT	等待（秒）
9	CALL	声明访问全局变量
10	TO	数组下标范围限定符
11	CONST	常量

2.3.2 输入输出

序号	关键字	说明
1	OUTPUT	输出
2	INPUT	输入
3	INTO	输入到变量

2.3.3 HTTP 客户端

序号	关键字	说明
1	GET	GET 请求
2	POST	POST 请求
3	PUT	PUT 请求
4	PATCH	PATCH 请求
5	DELETE	DELETE 请求
6	WITH	携带数据
7	SAVE	保存响应

2.3.4 HTTP 服务端

序号	关键字	说明
1	CREATESERVER	创建服务器
2	ROUTE	路由
3	RESPONSE	响应
4	FILE	文件响应
5	STATIC	静态目录

2.3.5 字面量

序号	关键字	说明
1	true	布尔真
2	false	布尔假

2.3.6 查询语句

序号	关键字	说明
1	TYPE	查询值的类型（语句）

2.4 运算符

类型	运算符
算术	+、-、*、/、%
比较	==、!=、>、<、>=、<=
赋值	=

2.5 分隔符

`() {} , ; "[]`

2.6 字面量

类型	示例
整数	123, -456
浮点数	3.14, -0.5
字符串	"hello", "你好"
布尔	true, false

2.7 转义序列

序列	含义
<code>\NL</code>	换行符
<code>\TAB</code>	制表符
<code>\"</code>	双引号
<code>\\</code>	反斜杠

3. 类型系统

3.1 支持的类型

类型	描述	示例
null	空值	未初始化变量的默认值
number	64 位浮点数	10, 3.14
string	字符串	"hello"
bool	布尔值	true, false
array	数组, 可存储任意类型的元素	[10, 20, 30]

3.2 类型转换

(1) 字符串拼接转换规则

左操作数类型	右操作数类型	结果类型	转换规则
string	string	string	直接拼接
string	number	string	数字→十进制字符串
string	bool	string	true → "true", false → "false"
string	null	string	null→"null"
number	string	string	数字 → 十进制字符串
number	number	number	数值相加
number	bool	number	bool → number: true → 1, false→0
number	null	number	null→0
bool	string	string	true → "true", false → "false"
bool	number	number	bool → number: true → 1, false→0

bool	bool	number	两者都转数字后相加
bool	null	number	bool 转数字, null→0
null	string	string	null→"null"
null	number	number	null→0
null	bool	number	null→0, bool 转数字
null	null	number	0 + 0 = 0

示例说明：

"value:" + 100 → "value:100"

"flag:" + true → "flag:true"

"empty:" + null → "empty:null"

100 + "px" → "100px"

注意： 只有当操作数中至少有一个为字符串时，才执行字符串拼接转换。两个数字相加仍为数值运算。

（2）比较运算转换规则

当使用比较运算符（==、!=、>、<、>=、<=）时，遵循以下转换规则：

比较类型	转换规则	说明
相同类型比较	直接比较	按类型自身规则比较
数字 vs 字符串	将字符串转换为数字	转换失败则按字符串比较
布尔 vs 非布尔	将布尔转换为数字	true→1, false→0
null 比较	null 只等于 null	与任何非 null 比较均为 false

具体转换行为：

- ① 相等比较 ==

数字与字符串：字符串转换为数字后比较

布尔与数字：true 转换为 1，false 转换为 0

布尔与字符串：布尔转换为 "true"/"false" 后比较

null 与 null: true

null 与非 null: false

② 不等比较 !=

逻辑与 == 相反

③ 大小比较 >、<、>=、<=

两个操作数均为数字：数值比较

一个为数字，一个为字符串：字符串转换为数字后比较

两个均为字符串：字典序比较

布尔参与比较：转换为数值（true=1，false=0）

null 参与比较：null 转换为 0

示例说明：

5 == "5" → true（字符串转数字）

5 == 5 → true

true == 1 → true（布尔转数字）

true == "true" → true（布尔转字符串）

null == null → true

null == 0 → false

5 > "3" → true（字符串转数字）

"10" > "2" → false（字典序比较："1" vs "2"）

true > 0 → true（true→1）

注意：类型转换仅在比较时临时进行，不改变操作数本身的类型。

4. 变量

4.1 变量与数组的定义

```
DEFINE 变量名;           !! 定义为 null

DEFINE 变量名 = 表达式;   !! 定义并赋值

DEFINE 变量名 维度声明 = 数组初始化;
```

4.1.1 维度声明

语法	说明	示例
[N]	从 0 开始，长度为 N	[3] → 下标 0, 1, 2
[N TO M]	从 N 开始，到 M 结束	[1 TO 3] → 下标 1, 2, 3
[]	自动推导长度（从 0 开始）	[] → 由初始化数据决定
[N][M]	多维数组	[2][3] → 2 行 3 列

4.1.2 数组初始化

```
{ 元素 1, 元素 2, 元素 3, ... }
```

示例

```
!! 一维数组
DEFINE a[3] = {10, 20, 30};
DEFINE a[1 TO 3] = {10, 20, 30};
DEFINE a[] = {10, 20, 30, 40};           !! 自动推导为 [4]

!! 二维数组
DEFINE b[2][3] = {1, 2, 3}, {4, 5, 6};
DEFINE b[1 TO 2][0 TO 2] = {1, 2, 3}, {4, 5, 6};

!! 三维数组
DEFINE c[2][2][2] = {1, 2}, {3, 4}, {5, 6}, {7, 8};

!! 空数组
DEFINE empty[3] = {};
```


规则说明

规则	说明
维度	由[]的数量决定
元素数量	初始化元素数量 ≤ 声明长度
元素不足	不足部分自动填充 null
元素过多	报错
自动推导	[] 表示由初始化数据决定长度
类型混合	同一数组可存储不同类型

4.2 变量赋值

变量名 = 表达式;

数组名 [索引表达式] = 表达式; !! 数组元素赋值

4.3 变量命名规则

- (1) 以字母或下划线开头
- (2) 后跟字母、数字、下划线
- (3) 不能使用关键字
- (4) 大小写敏感

4.4 作用域

- (1) 全局作用域：顶层变量
- (2) 函数作用域：函数内定义的变量
- (3) 块级作用域：IF/WHILE 块内定义变量不影响外部

4.5 全局变量访问

在函数内部，若定义了与全局变量同名的局部变量，全局变量将被隐藏。

如需在函数内访问被隐藏的全局变量，需使用 CALL 关键字声明。

语法格式：

```
CALL 变量名;
```

声明后，可通过 `@变量名` 的形式访问该全局变量。

示例：

```
DEFINE count = 10;      !! 全局变量

MAIN FUNCTION() {

    DEFINE count = 20;  !! 局部变量

    CALL count;         !! 声明访问全局 count

    OUTPUT count;       !! 输出 20（局部变量）

    OUTPUT @count;      !! 输出 10（全局变量）

}
```

约束：

- （1）CALL 只能在函数体内使用
- （2）CALL 的变量必须已在全局作用域中定义
- （3）未使用 CALL 声明时，无法通过 `@` 访问全局变量

5. 表达式

5.1 字面量表达式

- （1）数字：10, 3.14
- （2）字符串："hello"
- （3）布尔：true, false

5.2 标识符表达式

引用变量：x, name

5.3 二元表达式

运算符	优先级	结合性
* / %	最高	左结合
+ -	中	左结合
> < >= <=	低	左结合
== !=	最低	左结合

5.4 一元表达式

-表达式：数值取负

5.5 括号表达式

(表达式)

5.6 调用表达式

函数名(参数 1, 参数 2, ...)

5.7 数组访问表达式

数组变量名 [索引表达式][索引表达式]...

5.8 数组字面量表达式

{ 元素 1, 元素 2, 元素 3, ... }

示例：

```
!! 定义数组

DEFINE a[3] = {10, 20, 30};           !! 下标： 0, 1, 2

DEFINE c[1 TO 3] = {10, 20, 30};     !! 下标： 1, 2, 3

!! 访问数组

OUTPUT a[0];           !! 输出 10
```

```
OUTPUT a[1];           !! 输出 20

OUTPUT a[2];           !! 输出 30

OUTPUT c[1];           !! 输出 10

OUTPUT c[3];           !! 输出 30

OUTPUT c[0];           !! 报错：索引 0 超出范围 [1, 3]

!! 多维数组

DEFINE b[2][3] = {1,2,3}, {4,5,6};

OUTPUT b[0][1];        !! 输出 2

OUTPUT b[1][2];        !! 输出 6
```

6. 语句

类型	功能语句	非功能语句
语义作用	执行计算、I/O、控制流等运行时操作	声明标识符、定义实体、建立作用域
生效时机	运行时	编译时/解析时
顶层允许	禁止	允许
典型语法	OUTPUT、IF、WHILE	DEFINE、FUNCTION、MAIN

6.1 程序结构

程序由语句序列组成，每条语句以分号 ； 结尾。

功能必须写在函数内部。

6.2 块语句

```
{

    语句 1;

    语句 2;
```

```
}
```

6.3 变量定义语句

语法格式：

```
DEFINE 变量名;                !! 定义变量，初始值为 null
```

```
DEFINE 变量名 = 表达式;        !! 定义变量并赋值
```

```
DEFINE CONST 常量名 = 表达式;  !! 定义常量（必须初始化）
```

约束：

（1）常量必须在定义时初始化，不能单独声明

（2）常量一旦定义，其值不可改变

（3）同一作用域内常量名与变量名不能重名

（4）数组也可以定义为常量，语法为 `DEFINE CONST 数组名 维度声明 = 数组初始化`

示例：

```
DEFINE x;                      !! 变量，值为 null
```

```
DEFINE x = 10;                 !! 变量，值为 10
```

```
x = 20;                        !! 可以修改
```

```
DEFINE CONST PI = 3.14;        !! 常量，值为 3.14
```

```
PI = 3.14159;                 !! 报错：常量不能被赋值
```

```
DEFINE CONST A;                !! 报错：常量必须初始化
```

```
DEFINE CONST ARR[3] = {1, 2, 3}; !! 常量数组
```

```
ARR[0] = 10;                   !! 报错：常量数组元素不能被修改
```

6.4 赋值语句

```
x = 20;
```

6.5 输出语句

OUTPUT 表达式;

输出表达式的值到标准输出（不自动换行）。

6.6 输入语句

INPUT "提示文字" INTO 变量;

INPUT INTO 变量; !! 无提示

从标准输入读取一行字符串存入变量。

6.7 条件语句

```
IF (条件){  
    语句;  
}
```

```
IF (条件){  
    语句;  
} ELSE {  
    语句;  
}
```

条件必须是布尔类型。

6.8 循环语句

```
WHILE (条件){  
    语句;  
}
```

条件必须是布尔类型。

6.9 WAIT 语句

语法格式：

WAIT 表达式;

等待指定的秒数，表达式必须是数值类型。**WAIT** 语句会阻塞当前执行线程。

示例：

```
WAIT 5;      !! 等待 5 秒
```

```
WAIT 0.5;    !! 等待 0.5 秒
```

```
DEFINE delay = 3;
```

```
WAIT delay;  !! 等待 3 秒
```

6.10 函数定义语句

函数在 **BPS** 中是一个容器

函数定义的作用是：

- (1) 给一段代码起一个名字
- (2) 在调用时执行代码块

```
FUNCTION 函数名(参数 1, 参数 2){
```

```
    语句;
```

```
    RETURN 表达式;
```

```
}
```

参数是可选的

RETURN 语句可选，无返回值时返回 **null**

6.11 返回语句

```
RETURN 表达式;
```

只能在函数体内使用。

6.12 主函数

```
MAIN FUNCTION() {
```

```
    语句;

}
```

程序入口，必须有且仅有一个。

6.13 HTTP 请求语句

语法格式

- (1) GET "URL" SAVE 变量;
- (2) POST "URL" WITH 数据 SAVE 变量;
- (3) PUT "URL" WITH 数据 SAVE 变量;
- (4) PATCH "URL" WITH 数据 SAVE 变量;
- (5) DELETE "URL" SAVE 变量;

参数说明

参数	说明	适用方法
"URL"	请求地址,字符串字面量或表达式	所有方法
WITH 数据	携带的请求数据（JSON 字符串或表单数据）	POST、PUT、PATCH
SAVE 变量	将响应结果保存到变量	所有方法

示例代码

```
MAIN FUNCTION()

{

    DEFINE response;

    DEFINE userId = 123;

    !! 1. GET 请求

    GET "https://httpbin.org/get?id=" + userId SAVE response;
```



```
    OUTPUT "GET 结果: " + response + "\n\n";

    !! 2. POST 请求（带数据）

    POST "https://httpbin.org/post" WITH "{\"name\":\" 张 三 \",\"age\":25}" SAVE
response;

    OUTPUT "POST 结果: " + response + "\n\n";

    !! 3. PUT 请求（带数据）

    PUT "https://httpbin.org/put" WITH "{\"id\":1,\"status\":\"updated\"}" SAVE
response;

    OUTPUT "PUT 结果: " + response + "\n\n";

    !! 4. PATCH 请求（带数据）

    PATCH "https://httpbin.org/patch" WITH "{\"age\":26}" SAVE response;

    OUTPUT "PATCH 结果: " + response + "\n\n";

    !! 5. DELETE 请求

    DELETE "https://httpbin.org/delete?id=1" SAVE response;

    OUTPUT "DELETE 结果: " + response + "\n\n";

}
```

注意: WITH 关键字只能用于 POST、PUT、PATCH

数据通常是 JSON 字符串格式

SAVE 关键字用于保存响应, 变量必须先 DEFINE

6.14 服务器创建语句

```
CREATESERVER 端口号 {
```

```
ROUTE "路径" {  
  
    RESPONSE 表达式;  
  
}  
  
ROUTE "路径" {  
  
    FILE 表达式;  
  
}  
  
STATIC "目录";  
  
}
```

服务器在后台运行，不阻塞主程序

RESPONSE：返回字符串响应

FILE：返回文件内容

STATIC：静态文件目录

6.15 数组定义语句

语法格式

```
DEFINE 变量名 维度声明 = 数组初始化;
```

参数说明

参数	说明
变量名	数组名称
维度声明	一个或多个 [N] 或 [N TO M] 或 []
数组初始化	{元素 1, 元素 2, ...} 或多个 {} 组合

6.16 数组访问语句

读取元素

```
数组名 [ 索引表达式 ];
```

修改元素

数组名 [索引表达式] = 表达式;

示例

```
DEFINE a[3] = {10, 20, 30};

OUTPUT a[0];           !! 读取: 输出 10

a[0] = 100;           !! 修改

OUTPUT a[0];           !! 输出 100
```

```
DEFINE b[2][3] = {1,2,3}, {4,5,6};

OUTPUT b[0][1];        !! 输出 2

b[1][2] = 100;         !! 修改

OUTPUT b[1][2];        !! 输出 100
```

```
!! 自定义起始下标

DEFINE c[1 TO 3] = {10, 20, 30};

OUTPUT c[1];           !! 输出 10

c[3] = 100;            !! 修改

OUTPUT c[3];           !! 输出 100
```

6.17 TYPE 语句

语法格式:

```
TYPE(表达式);
```

功能:

获取并输出表达式的类型名称。

返回值:

类型	返回值
null	"null"

number	"number"
string	"string"
bool	"bool"
array	"array"

- 注意：
- （1）`TYPE` 是语句，不是函数，不能作为表达式的一部分使用
 - （2）`TYPE` 只能用于查询类型，不能用于类型转换

示例：

```
DEFINE x = 10;  
  
TYPE(x);    !! 返回： number
```

7. 内置函数

7.1 无内置函数

BPS-26 标准不提供内置函数。所有功能通过关键字语句实现。

8. 程序执行

8.1 执行入口

- （1）解析所有 `FUNCTION` 定义
- （2）查找 `MAIN FUNCTION`
- （3）执行 `MAIN FUNCTION` 中的语句
- （4）等待所有 `CREATESERVER` 服务器退出

8.2 执行方式

bps.exe 脚本文件.bps

9. 错误处理

9.1 错误类型

类型	描述
Syntax Error	词法/语法错误
Runtime Error	运行时错误（除零、未定义变量等）
Type Error	类型不匹配

9.2 错误报告格式：

BPS <错误类型>

File(文件): <文件名>

Line(行): <行号>

Column(列): <列号>

<源代码行>

^

<错误描述>

Hint(提示): <修复建议>

10. 实现限制

10.1 已知限制

- (1) 不支持对象/字典类型
- (2) 不支持标准库导入
- (3) 字符串最大长度：无硬性限制（受内存限制）

10.2 未实现特性

无

附录 A：完整语法示例

!! 变量定义

```
DEFINE name = "BPS-26";
```

```
DEFINE version = 1.0;
```

```
DEFINE const = 10;
```

!! 常量定义

```
DEFINE CONST PI = 3.14159;
```

```
DEFINE CONST APP_NAME = "BPS";
```

!! 数组定义

```
DEFINE arr[3] = {1, 2, 3};
```

```
DEFINE matrix[2][3] = {1,2,3}, {4,5,6};
```

```
DEFINE custom[1 TO 3] = {10, 20, 30};
```

```
DEFINE auto[] = {1, 2, 3, 4, 5};
```

!! 函数定义

```
FUNCTION add(a, b) {
```

```
    RETURN a + b;
```

```
}
```

!! 主函数

MAIN FUNCTION() {

!! 数组访问

OUTPUT arr[0]; !! 输出 1

arr[1] = 100; !! 修改元素

OUTPUT arr[1]; !! 输出 100

OUTPUT matrix[0][1]; !! 输出 2

matrix[1][2] = 99; !! 修改元素

OUTPUT matrix[1][2]; !! 输出 99

OUTPUT custom[1]; !! 输出 10（从 1 开始）

OUTPUT custom[3]; !! 输出 30

OUTPUT auto[0]; !! 输出 1（自动推导长度）

OUTPUT auto[4]; !! 输出 5

!! HTTP 请求

GET "https://api.example.com/data" SAVE response;

OUTPUT response;

!! HTTP 服务器

CREATESERVER 8080 {

ROUTE "/" {

 RESPONSE "Hello " + name;

}

```
        ROUTE "/data" {  
            FILE "data.json";  
        }  
        STATIC "./static";  
    }  
  
    OUTPUT "BPS-26 标准启动\nl";  
  
    DEFINE result = add(10, 20);  
  
    OUTPUT "10+20=" + result + "\nL";  
  
    DEFINE count = 20;  
  
    CALL count;  
  
    OUTPUT @count;  
  
    OUTPUT PI+"\nL"+APP_NAME;  
  
    WAIT 2; !! 等待 2 秒  
  
    OUTPUT "程序结束\nL";  
}
```

附录 B：标准符合性声明

本实现（Basic Programming Script 0.0.2.3）完全符合 BPS-26.2 标准的所有规定，无额外扩展功能，无未实现特性。

附录 C：Token 编号与符号检索表

当解析器报错显示 expected token type X 时，可查阅下表了解该编号对应的 Token 类型，以便快速定位语法问题，详细请看《BPS Token 编号与符号检索表》。

BPS 编程语言官方网站：

`bps.shdiv.net`

最新标准、版本更新、下载资源请访问官网。

至此文档结束。